Guía de estudio - Taller ICNO

Proyecto Final de Programación Orientada a Objetos · Isaac Navarro

Esta guía es para entender el proyecto completo y poder explicar cualquier línea en la revisión. Léela junto al código abierto. Todo el código está comentado línea por línea: si el profe apunta a un renglón, la respuesta ya está escrita a la derecha. Esta guía te da el hilo para unir esos renglones.

Orden de lectura: primero el guion de exposición, luego la idea general, la estructura de carpetas, cada archivo, los conceptos que más pregunta el profe, y al final las preguntas trampa con su respuesta.

Guion para la revisión (10 minutos)

Los números de línea son de `inc/taller.php` salvo que se diga otro archivo.

1. La idea en una frase. "Es un taller mecánico. Los clientes registran vehículos y piden cotizaciones. El precio sale de dos jerarquías de herencia: el tipo de servicio decide cómo se calcula el costo, y el tipo de vehículo da un factor que lo multiplica."

2. Las clases (R1). Abre `inc/taller.php` y muestra las 11 agrupadas:

Grupo	Clases	Línea
Familia de servicios	`Servicio` padre, `ServicioPreventivo` y `ServicioCorrectivo` hijas	28, 58, 75
Familia de vehículos	`Vehiculo` padre; `Auto`, `Camioneta`, `Camion`, `Motocicleta` hijas	95, 137 a 149
Apoyo	`Usuario`, `Cotizacion`	157, 182
Principal	`Taller`, la única que habla con MySQL	212

3. Encapsulamiento (R2). Línea 30: `private $id;`. Baja a los getters y setters. Di: "nadie cambia un precio directo, pasa por `setCostoBase()`; si mañana valido que no sea negativo, lo hago en un solo lugar". Remata con la línea 213: la conexión `$bd` de `Taller` es privada y ninguna página escribe SQL.

4. Herencia (R3). Línea 58: `extends Servicio`. Línea 61: `parent::__construct()`. Di: "le paso al padre lo que a él le toca y yo guardo lo mío, que es el kilometraje". Luego la pregunta que siempre cae: "¿cómo sabe el programa si una fila es preventivo o correctivo?". Línea 312 `servicioDesdeFila()` y línea 266 el `switch` de `vehiculoDesdeFila()`: la tabla tiene columna `tipo` y según eso se crea la clase hija correcta.

5. Polimorfismo (R4). Dos métodos, no uno:

Método	Padre	Hijas
`calcularCosto()`	42, devuelve el costo tal cual	66 lo multiplica por 0.9; 83 suma 250 por hora
`getFactor()`	115, devuelve 1.0	138, 142, 146, 150: 1.0, 1.3, 1.6, 0.7

Los dos se juntan en la **línea 365**:

```
$total += $s->calcularCosto() * $vehiculo->getFactor();
```

Di: "esta línea no pregunta de qué tipo es nada; PHP llama la versión correcta según la clase real de cada objeto". Ejemplo con números: correctivo de 1000 con 2 horas en una camioneta: (1000 + 500) × 1.3 = 1950.

6. CRUD (R5). Los cuatro están seguidos en `Taller` y cada uno es una consulta SQL: `agregarServicio()` 319 (INSERT), `getServicios()` 326 (SELECT), `modificarServicio()` 340 (UPDATE), `eliminarServicio()` 347 (DELETE). La página que los usa es `servicios.php`. Si piden demostrarlo, entra como admin y haz las cuatro operaciones en pantalla.

7. Demostración en vivo (3 minutos). Entra como cliente, registra una camioneta, cotiza un servicio correctivo y muestra el desglose con el factor 1.3. Luego entra como admin, edita el precio de ese servicio y muestra que la tabla cambia. Con eso viste R2 a R5 corriendo.

Qué hace el programa

Es un sistema web para un taller mecánico. Funciona en el navegador y guarda todo en una base de datos MySQL llamada `taller_icno`. Hay dos tipos de usuario:

- **Cliente.** Se registra, da de alta sus vehículos (auto, camioneta, camión o motocicleta) y pide cotizaciones eligiendo un vehículo y los servicios que quiere.
- **Administrador.** Administra el catálogo de servicios (el CRUD), ve todos los vehículos de los clientes y revisa las cotizaciones que llegan.

Hay dos tipos de servicio: preventivo (mantenimiento programado, con 10% de descuento) y correctivo (una reparación, que cobra 250 pesos por cada hora de mano de obra). Y cada tipo de vehículo tiene un factor que sube o baja el precio: una camioneta paga más que una moto. Ahí está la herencia y el polimorfismo, que es el corazón del proyecto.

Dónde se cumple cada requisito

El profe pide seis cosas. Aquí está dónde vive cada una:

1. **Cuatro clases o más.** Tenemos once, todas en `inc/taller.php`: `Servicio`, `ServicioPreventivo`, `ServicioCorrectivo`, `Vehiculo`, `Auto`, `Camioneta`, `Camion`, `Motocicleta`, `Usuario`, `Cotizacion` y `Taller`.
2. **Encapsulamiento.** Todos los atributos son `private` y solo se tocan con getters y setters. Míralo en cualquier clase.
3. **Herencia.** Dos jerarquías: `ServicioPreventivo` y `ServicioCorrectivo` extienden de `Servicio`; `Auto`, `Camioneta`, `Camion` y `Motocicleta` extienden de `Vehiculo`.
4. **Polimorfismo.** `calcularCosto()` se reescribe en cada tipo de servicio y `getFactor()` se reescribe en cada tipo de vehículo.
5. **CRUD.** La página `servicios.php` crea, lista, modifica y elimina servicios, y cada operación termina en una consulta SQL dentro de `Taller`.
6. **Diagrama UML.** Está en `documentacion/diagrama_uml.html`.

Estructura de carpetas

```
tallermecanico/
├── inc/taller.php        el motor: clases, conexión, sesión, roles, plantilla
├── css/estilos.css       todo el diseño
├── img/                  logos y fotos de servicios
├── documentacion/        esta guía, memoria técnica, UML, capturas
├── index.php, login.php, registro.php, logout.php        públicas
├── cliente.php, mis_autos.php, cotizar.php, mis_cotizaciones.php   cliente
└── panel.php, servicios.php, vehiculos.php, cotizaciones.php       admin
```

Las páginas se quedan en la raíz porque son las direcciones que abre el navegador. Las carpetas guardan lo que las páginas cargan: `inc/` el código que no se ve, `css/` el diseño, `img/` las imágenes. Si preguntan por qué `taller.php` está en `inc/`: "inc" viene de "include", es la convención para archivos que otros cargan con `require` y que nadie abre directo en el navegador.

La base de datos

Se llama `taller_icno` y tiene cinco tablas. Cada clase del modelo tiene su tabla, y las relaciones se hacen con llaves foráneas (`id_usuario`, `id_vehiculo`, etc.).

Tabla	Qué guarda	Clase que la representa
`usuarios`	id, nombre, correo, clave (cifrada), rol (`admin` o `cliente`)	`Usuario`
`vehiculos`	id, id_usuario, placas, marca, modelo, color, anio, kilometraje, tipo	`Vehiculo` y sus hijas

Tabla	Qué guarda	Clase que la representa
`servicios`	id, tipo (`preventivo` o `correctivo`), descripcion, costo_base, extra	`Servicio` y sus hijas
`cotizaciones`	id, id_usuario, id_vehiculo, total, estado, fecha	`Cotizacion`
`cotizacion_servicios`	id_cotizacion, id_servicio	la lista de servicios de una cotización

Dos detalles que conviene saber explicar:

- **La columna `tipo`** es la que dice de qué clase hija es cada fila. Cuando el programa lee la tabla, mira esa columna y crea el objeto correcto. Así se guarda la herencia en una tabla plana.
- **La columna `extra`** de servicios guarda el kilometraje si es preventivo o las horas de mano de obra si es correctivo. Es una sola columna para no tener dos tablas.

La conexión se hace con **PDO**, que es la librería estándar de PHP para bases de datos. Todas las consultas usan `prepare()` con signos `?` en vez de pegar los valores en el texto SQL. Eso evita la inyección SQL.

Los archivos, uno por uno

inc/taller.php

Es el archivo más importante. Todas las demás páginas lo cargan con `require 'inc/taller.php'` en su primera línea. Está dividido en tres partes:

- **Parte 1: las once clases** (líneas 28 a 418). Todo el modelo del taller.
- **Parte 2: base de datos, sesión y control de acceso** (líneas 425 a 470). Lee los datos de conexión desde `inc/config.php` (427), que no se sube a GitHub, se conecta a MySQL con PDO (433), arranca la sesión con `session_start()` (441) y crea un único objeto `$taller = new Taller($bd)` (443) que usan todas las páginas. También define dos funciones: `usuarioActual()` (446), que devuelve el `Usuario` logueado leyendo su id de `$_SESSION`, y `exigirRol($rol)` (456), que corta la página y redirige al login si nadie ha entrado o si el rol no es el correcto.
- **Parte 3: la plantilla** (477 en adelante). Dos funciones, `encabezado()` y `pie()`, que dibujan la barra de arriba y el pie de página. El menú de la barra cambia según quién esté usando el sistema: visitante, cliente o admin.

Las clases de inc/taller.php

`Servicio` (padre, línea 28). Tiene id, descripción y costo base. Su método `calcularCosto()` devuelve el costo base tal cual. `getTipo()` devuelve "General".

`ServicioPreventivo` (hija, 58). Agrega kilometraje. `calcularCosto()` devuelve el costo base por 0.9.

`ServicioCorrectivo` (hija, 75). Agrega horas de mano de obra. `calcularCosto()` devuelve costo base más 250 por cada hora.

`Vehiculo` (padre, 95). Tiene id, placas, marca, modelo, color, año y kilometraje. Su método `getFactor()` devuelve 1.0.

`Auto`, `Camioneta`, `Camion`, `Motocicleta` (hijas, 137 a 149). Solo reescriben `getFactor()` y `getTipo()`. Los factores son 1.0, 1.3, 1.6 y 0.7.

`Usuario` (157). Quién está usando el sistema: id, nombre, correo y rol. El método `esAdmin()` regresa verdadero si el rol es `admin`. No tiene setters porque el usuario no se edita desde el sistema.

`Cotizacion` (182). Junta un objeto `Vehiculo`, un arreglo de objetos `Servicio`, el total, el estado (`pendiente` o `revisada`) y la fecha.

`Taller` (clase principal, 212). Es la única que habla con la base de datos. Recibe la conexión PDO en su constructor y la guarda en el atributo privado `$bd`. Sus métodos se agrupan así:

- Usuarios: `registrarCliente()` 225, `verificarLogin()` 251, `buscarUsuarioPorId()`, `buscarUsuarioPorCorreo()`.
- Vehículos: `agregarVehiculo()` 275, `getVehiculosDeUsuario()`, `getTodosVehiculos()`, `buscarVehiculo()`.
- Servicios (el CRUD): `agregarServicio()` 319, `getServicios()` 326, `buscarServicio()`, `modificarServicio()` 340, `eliminarServicio()` 347.
- Cotizaciones: `crearCotizacion()` 357, `getCotizacionesDeUsuario()`, `getTodasCotizaciones()`, `marcarRevisada()` 415.

Además tiene tres métodos privados que convierten una fila de la base de datos en un objeto: `vehiculoDesdeFila()` 264, `servicioDesdeFila()` 312 y `cotizacionDesdeFila()`. Son importantes y se explican en la sección de herencia.

index.php

La pantalla de inicio para quien no ha entrado. Es una landing con un carrusel de servicios, tarjetas y botones que llevan a registrarse o iniciar sesión. No tiene lógica, solo presenta. El carrusel es JavaScript que corre en el navegador, no en PHP.

login.php, registro.php y logout.php

- **login.php.** Recibe correo y contraseña por POST y llama a `$taller->verificarLogin()`. Si regresa un `Usuario`, guarda su id en `$_SESSION['usuario_id']` y lo manda a `panel.php` si es admin o a `cliente.php` si es cliente.

- **registro.php.** Un cliente nuevo crea su cuenta. Llama a `$taller->registrarCliente()`, que guarda la contraseña cifrada con `password_hash()`. Si el correo ya existe, devuelve falso y se muestra el error. Al terminar queda con la sesión iniciada.
- **logout.php.** Destruye la sesión y regresa al inicio.

Páginas del cliente

Todas empiezan con `$u = exigirRol('cliente')`.

- **cliente.php.** Bienvenida con accesos rápidos y cuántos autos y cotizaciones tiene.
- **mis_autos.php.** Formulario para registrar un vehículo con todas sus características y un menú para elegir el tipo. Según el tipo, hace `new Auto(...)`, `new Camioneta(...)`, `new Camion(...)` o `new Motocicleta(...)` (línea 28) y lo manda a `$taller->agregarVehiculo()`. Abajo lista sus vehículos y en la columna Tipo muestra `$v->getTipo()`.
- **cotizar.php.** El cliente elige uno de sus autos y marca los servicios con casillas. Al enviar, llama a `$taller->crearCotizacion()` (línea 25) y luego muestra el desglose: cada servicio con su subtotal y el total. La línea 34 repite el cálculo polimórfico para pintarlo en pantalla. Aquí es donde mejor se ve el polimorfismo funcionando.
- **mis_cotizaciones.php.** Lista las cotizaciones del cliente con su vehículo, servicios, total y estado.

Páginas del administrador

Todas empiezan con `$u = exigirRol('admin')`.

- **panel.php.** Resumen con cuántos servicios, vehículos y cotizaciones hay, y cuántas cotizaciones están pendientes.
- **servicios.php.** Es la página del CRUD. Es la más larga porque hace las cuatro operaciones:
  - **Crear** (línea 23): el formulario manda un campo oculto `accion=crear`. Según el tipo elegido se crea un `ServicioPreventivo` o un `ServicioCorrectivo` (35) y se pasa a `agregarServicio()`, que hace el `INSERT`.
  - **Consultar:** `getServicios()` hace el `SELECT` y la tabla recorre la lista mostrando `$s->getTipo()` y `$s->calcularCosto()`.
  - **Actualizar** (44): el botón Editar manda a `servicios.php?editar=2` (68). Se busca ese servicio, se llena el formulario con sus datos, y al guardar llega `accion=actualizar` y se llama a `modificarServicio()`, que hace el `UPDATE`.
  - **Eliminar** (59): el botón Eliminar manda a `servicios.php?eliminar=2` y `eliminarServicio()` hace el `DELETE`.

Fíjate cómo se distingue qué acción hacer: `$_POST['accion']` para los formularios, y `$_GET['editar']` y `$_GET['eliminar']` para los enlaces. GET son datos que van en la dirección; POST son datos de un formulario.

- **vehiculos.php.** Solo consulta. Muestra todos los vehículos de todos los clientes con el nombre del dueño, usando un `JOIN` con la tabla usuarios.
- **cotizaciones.php.** Muestra todas las cotizaciones con el nombre del cliente. El enlace Marcar revisada manda `?revisar=5` y llama a `marcarRevisada()`, que cambia el estado con un `UPDATE`.

css/estilos.css

Es el diseño. Define los colores, la tipografía, las tarjetas, las tablas, los formularios, las etiquetas de estado y el carrusel. Al inicio hay variables de color (por ejemplo `--naranja`) que se usan en todo el archivo.

Los conceptos que hay que dominar

Encapsulamiento

Quiere decir que los datos de un objeto están protegidos. En vez de dejar que cualquiera cambie el precio directo, lo marco `private` y obligo a pasar por un método:

```
private $costoBase;                              // nadie lo toca directo
public function getCostoBase() { return $this->costoBase; }   // para leerlo
public function setCostoBase($costoBase) { $this->costoBase = $costoBase; } // para
cambiarlo
```

La ventaja: si algún día quiero validar el precio (por ejemplo, que no sea negativo), lo hago en el setter y queda protegido en un solo lugar.

`$this` significa "este objeto". Dentro de un método, `$this->costoBase` es el costoBase del objeto sobre el que se llamó el método. En Java es `this`, en PHP es `$this`.

El mejor ejemplo de encapsulamiento en el proyecto es la clase `Taller`: su conexión `$bd` es privada y nadie fuera de ella escribe SQL. Las páginas solo llaman métodos como `getServicios()` sin saber cómo está hecha la consulta.

Herencia

Es cuando una clase toma todo lo de otra y le agrega lo suyo. `ServicioPreventivo` hereda de `Servicio`, así que ya tiene id, descripción, costo base y todos los getters sin volver a escribirlos. Solo agrega el kilometraje.

```
class ServicioPreventivo extends Servicio {
    private $kilometraje;
    public function __construct($id, $descripcion, $costoBase, $kilometraje) {
        parent::__construct($id, $descripcion, $costoBase); // le pasa lo comun al padre
        $this->kilometraje = $kilometraje;                  // guarda lo suyo
    }
}
```

`parent::__construct(...)` llama al constructor del padre para que él guarde los datos que le tocan. Así no repito ese código. Es el equivalente de `super(...)` en Java.

La segunda jerarquía es más sencilla. Las cuatro hijas de `Vehiculo` no agregan atributos, solo cambian el comportamiento:

```
class Camioneta extends Vehiculo {
    #[\Override] public function getFactor() { return 1.3; }
    #[\Override] public function getTipo()   { return "Camioneta"; }
}
```

Cómo se guarda la herencia en la base de datos. Una tabla no sabe de clases. Por eso la tabla `vehiculos` tiene la columna `tipo`. Cuando `Taller` lee una fila, el método privado `vehiculoDesdeFila()` (línea 264) mira esa columna y crea el objeto de la clase correcta:

```
switch ($f['tipo']) {
    case 'camioneta': return new Camioneta(...$args);
    case 'camion':    return new Camion(...$args);
    case 'moto':      return new Motocicleta(...$args);
    default:          return new Auto(...$args);
}
```

`servicioDesdeFila()` (312) hace lo mismo con la columna `tipo` de servicios. Esta es una pregunta típica: "¿cómo sabe el programa si un servicio es preventivo o correctivo si todo está en la misma tabla?". La respuesta es esa columna y ese método.

Polimorfismo

Es la parte más importante y la que más van a preguntar. La misma llamada da un resultado distinto según el tipo real del objeto. En el proyecto hay dos métodos polimórficos:

`calcularCosto()` en la familia de servicios:

- En `Servicio` (padre, 42) devuelve el costo base tal cual.
- En `ServicioPreventivo` (66) devuelve el costo base con 10% menos.
- En `ServicioCorrectivo` (83) devuelve el costo base más 250 por cada hora.

```
// En ServicioPreventivo
public function calcularCosto() { return $this->getCostoBase() * 0.9; }

// En ServicioCorrectivo
public function calcularCosto() { return $this->getCostoBase() + ($this->horasManoObra *
250); }
```

`getFactor()` en la familia de vehículos (115 y 138 a 150): Auto 1.0, Camioneta 1.3, Camión 1.6, Motocicleta 0.7.

Los dos se juntan en una sola línea dentro de `crearCotizacion()`, línea 365:

```
$total += $s->calcularCosto() * $vehiculo->getFactor();
```

Fíjate que no pregunta de qué tipo es el servicio ni de qué tipo es el vehículo. PHP llama la versión correcta según la clase real de cada objeto. Eso es polimorfismo. Ejemplo con números: un servicio correctivo de costo base 1000 con 2 horas, en una camioneta, da (1000 + 500) × 1.3 = 1950.

El `#[\Override]` que está arriba de esos métodos es una marca que le dice a PHP "esto reescribe un método del padre". Si me equivoco al escribir el nombre, PHP me avisa. Es el equivalente de `@Override` en Java.

El CRUD

CRUD son las cuatro operaciones básicas sobre los datos. Cada una es un método de `Taller` que ejecuta una consulta SQL:

- **Create (crear):** `agregarServicio()` 319 hace un `INSERT INTO servicios`.
- **Read (consultar):** `getServicios()` 326 hace un `SELECT * FROM servicios`.
- **Update (actualizar):** `modificarServicio()` 340 hace un `UPDATE servicios`.
- **Delete (eliminar):** `eliminarServicio()` 347 hace un `DELETE FROM servicios`.

La página `servicios.php` las usa. También hay CRUD parcial en otras tablas (vehículos se crean y consultan, cotizaciones se crean, consultan y actualizan su estado), pero el completo es el de servicios.

Sesión y control de acceso

En una página web cada clic es una petición nueva y el programa arranca de cero. Para saber quién está usando el sistema, al hacer login se guarda solo el id del usuario en `$_SESSION['usuario_id']`. En cada página, `usuarioActual()` (446) toma ese id y pide el `Usuario` completo a la base de datos.

`exigirRol('admin')` o `exigirRol('cliente')` (456) va al inicio de cada página protegida. Si nadie ha entrado, redirige al login. Si entró pero con el rol equivocado, lo manda a su propia zona. Por eso un cliente no puede abrir `servicios.php` aunque escriba la dirección a mano.

Las contraseñas nunca se guardan en texto plano: `password_hash()` (229) las cifra al registrar y `password_verify()` (255) las compara al entrar.

Preguntas trampa y cómo responderlas

- **"¿Por qué usaste base de datos si no la pedí?"** Para que los datos no se pierdan al cerrar el navegador. Pero la POO no depende de eso: `Taller` convierte cada fila en objeto, así que el resto del sistema trabaja con objetos igual que si estuvieran en un arreglo.
- **"¿Por qué PHP y no Java?"** El profe acepta otros lenguajes si se cumplen los seis requisitos. PHP tiene clases, `private`, `extends`, `parent::` y `#[\Override]`, que son los equivalentes directos de Java. Y me permitió hacerlo web, que es como se usan estos sistemas en la realidad.
- **"¿Dónde está el `this`?"** En PHP es `$this`. Línea 36 en el constructor de `Servicio`, y en todos los setters.
- **"¿Por qué `Usuario` no tiene setters?"** Porque el usuario no se edita desde el sistema. Solo se crea y se consulta. Encapsulamiento también es no exponer lo que no hace falta.

- **"¿Esto de `getTipo()` también es polimorfismo?"** Sí. Cada hija devuelve su nombre y lo uso en las etiquetas de colores de las tablas. Pero el polimorfismo central es `calcularCosto()` y `getFactor()`.
- **"¿Qué es `instanceof`?"** Línea 320. Pregunta de qué clase es un objeto. Lo uso en `agregarServicio()` para saber si guardar el extra como kilometraje o como horas.
- **"¿Por qué `?` en las consultas SQL?"** Para que los datos del usuario nunca se peguen dentro del texto SQL. Evita inyección SQL. PDO los manda por separado.
- **"¿Qué pasa si un cliente escribe `servicios.php` en la barra?"** Línea 13 de `servicios.php`: `exigirRol('admin')` lo detecta y lo manda a `cliente.php`.
- **"Si creo un correctivo de 1000 con 3 horas, ¿cuánto cobra?"** Solo: 1000 + 3 × 250 = 1750. En auto 1750, en camioneta 2275, en camión 2800, en moto 1225.
- **"¿Dónde está la contraseña de la base de datos?"** En `inc/config.php`, un archivo aparte que está en `.gitignore`. El código público solo tiene `config.ejemplo.php` con valores de muestra. Así el repositorio no expone credenciales.
- **"¿Por qué el id es 0 al crear un objeto nuevo?"** Línea 35 de `servicios.php`. Todavía no existe en la base; MySQL le asigna el id real al hacer el `INSERT`. El 0 es un marcador de "aún no guardado".

Preguntas de práctica

Respóndelas tú antes de la revisión. Si puedes con estas, puedes con la clase.

1. ¿Por qué los atributos son `private` y no `public`?
2. ¿Qué hace `parent::__construct()` y por qué lo uso?
3. Si creo un servicio correctivo con costo base 1000 y 3 horas, ¿cuánto cobra en un auto y cuánto en un camión? ¿Por qué?
4. ¿Dónde está el polimorfismo y cómo lo demuestro en pantalla?
5. ¿Cómo sabe el programa si una fila de la tabla `vehiculos` es un `Auto` o una `Camioneta`?
6. ¿Qué diferencia hay entre `$_POST` y `$_GET` en este proyecto?
7. ¿Qué hace `exigirRol()` y qué pasa si un cliente intenta entrar a `servicios.php`?
8. ¿Por qué las consultas usan `prepare()` con `?` en vez de pegar los valores en el SQL?
9. Enséñame en el código dónde están las cuatro operaciones del CRUD y qué consulta SQL ejecuta cada una.
10. ¿Qué clase es la única que habla con la base de datos y por qué está bien que sea así?

Cómo practicar

Abre `inc/taller.php` y tapa los comentarios de la derecha con la mano. Lee la línea y di en voz alta qué hace. Si no sabes, destapa. Empieza por las líneas 28 a 90 (las dos clases de servicio) y 355 a 375

(`crearCotizacion`), que son las que más te van a preguntar. Luego haz la demostración en vivo del guion dos veces sin leer.

Cómo correr el proyecto

Está explicado a detalle en `como_correr.md`. En corto: necesitas PHP y MySQL con la base `taller_icno` creada. Desde la carpeta `tallermecanico` corres `php -S localhost:8000` y abres `http://localhost:8000`. También está subido y funcionando en `https://taller.navmac.com` (también responde por la IP `http://148.113.167.5`).